发送使用一个 32 字节的软件环形缓冲区：`uartputc()` 把字符放进缓冲区，缓冲区满时睡眠；`uartstart()` 在发送 FIFO 有空位时搬字符过去，并打开“发送空”中断；`uartintr()` 读空接收 FIFO、把字符交给 `consoleintr()`，再继续发送。缓冲区空了就关掉发送中断，否则会一直触发。内核 `printf` 在 panic 等场合使用同步的 `uartputc_sync()`，不依赖中断。

### 7.3.2 控制台与行规程

`consoleintr()` 实现了最基本的行规程：回显、退格、整行缓冲、`Ctrl-D` 结束输入、`Ctrl-P` 打印进程表、`Ctrl-C` 向前台进程组发送 SIGINT。读进程在 `consoleread()` 里睡眠，直到收到一整行。

### 7.3.3 TTY 层

`/dev/console`、`/dev/ttyS0`、`/dev/tty` 共用 Mini UART 这一个硬件后端，但含义不同：

- `/dev/console` 是系统控制台，内核日志直接写 UART，不经过文件系统；
- `/dev/ttyS0` 是具体的串口终端，`init` 在上面启动 `login`；
- `/dev/tty` 是“当前进程的控制终端”，按调用者的 `ctty` 动态转发：串口会话转到 `ttyS0`，SSH 会话转到对应的 PTY 从端。

`login` 调用 `tty_attach()` 成为新会话和进程组的组长，并把 `ttyS0` 设为控制终端；之后 fork 出的 shell 和命令继承会话、进程组和控制终端。shell 运行外部命令时把它设为前台进程组，`Ctrl-C` 产生的 SIGINT 只发给这个组，所以能终止 `ping`、`tftp` 或整条管道，而不会杀掉 shell 和 `login`。

一个由此暴露的问题是：旧版 `ps` 通过系统调用让内核 `printf` 直接打印进程表，结果在 SSH 会话里执行 `ps`，输出却出现在串口上。内核 `printf` 写的是系统控制台，不看调用者的 fd 1。现在 `sys_ps()` 把快照格式化到用户缓冲区，由 `ps` 自己 `write(1, ...)`。

## 7.4 USB 与 HID 键盘

USB 键盘是本项目里协议层次最多的驱动：从主机控制器的寄存器，到 Hub、事务转换器、描述符、HID 报告，再到输入子系统和控制台。本节沿着“插上键盘—识别—配置—读按键”的顺序，把每一层讲清楚：先看硬件路径和三层软件的分工，再看一次事务在总线上如何由 Token、Data、Handshake 三相包组成，然后顺着描述符与 control 传输完成枚举，然后处理高速 Hub 后面低速设备的 split 时序，接着用 URB 和中断把一份按键报告送到输入子系统与 `/dev/input/eventN`，最后是真机上从 STALL 走到第一份报告的九个阶段。Hub 中断驱动的热插拔、复合 HID 接口和驱动模块的自动装载放在第 9 章。

### 7.4.1 三层软件与硬件路径

USB 驱动分三层：

- **HCD**（`kernel/dwc2.c`）：复位 DWC2 控制器、强制主机模式、开 DMA；给根端口和 Hub 端口上电、复位；用 host channel 发起 control、bulk、interrupt 传输；
- **USB core**（`kernel/usb.c`）：`struct usb_device`、按 ID 或接口类别匹配驱动、Linux 风格的 URB 接口；
- **类驱动**：HID 键盘（`usbkbd.c`）、CDC-ECM 网卡（`usbnet.c`，用于 QEMU）、MT7601U 无线网卡（`mt7601u.c`）。

一份按键从键盘走到用户进程，要穿过这样一条链：

```
USB 键盘（低速 1.5 Mb/s）
  -> LAN951x 高速 Hub（TT 完成速率转换）
  -> DWC2 主机控制器（ch6 interrupt-IN + DMA）
  -> USB bus：struct usb_device
  -> HID Boot Keyboard 类驱动（usbkbd.c）
```